Bi-Encoder Retrieval vs. Cross-Encoder Re-Ranking

A RAG stress test on the FIA 2026 Formula 1 Technical Regulations

Aviral Tyagi · 2025201086

`github.com/XProtocol0/rag-baseline-vs-reranking`

1 Setup

Document. *FIA 2026 F1 Regulations, Section C [Technical], Issue 20* (254 pages, about 100k words). Articles C1–C18, the Definitions appendix and Appendix C5 were indexed (204 pages). The table of contents and the CAD/table appendices were excluded.

Preprocessing.

- Removed the 10-line running header on every page and the governance boilerplate.
- Repaired a font defect that extracts the "ff" ligature as the letter `a` ("effect" → *eaect*).
- **Clause-aware chunking**: split at clause numbers (C5.1.3), never mix two sections, at most 120 words, and a section-title header (e.g. *C9.7 Reverse gear*) on chunks that start mid-section.

The result is **1,032 chunks**. At 120 words only 0.4% of chunks exceed the bi-encoder's 256-token window (5.6% would at 150 words), so the baseline is not handicapped by truncation.

Pipelines. Both pipelines share one corpus, one candidate set and one prompt:

- **Pipeline 1 (baseline):** `all-MiniLM-L6-v2` bi-encoder, FAISS `IndexFlatIP` (exact cosine) → top 10 → top 3 → LLM (Answer 1).
- **Pipeline 2 (re-ranking):** the *same* top 10 → `cross-encoder/ms-marco-MiniLM-L-6-v2` → re-sort → top 3 → LLM (Answer 2).

Generator. DeepSeek `deepseek-flash` at temperature 0, with thinking disabled. The prompt says to answer *only* from the excerpts and to cite them; this stops the LLM from hiding retrieval failures with what it already knows about F1. Assertions in the notebook check that Pipeline 2 re-ranks exactly Pipeline 1's candidates.

2 Architecture comparison

	Bi-encoder	**Cross-encoder**
Input	Query and chunk encoded *separately*	`[CLS] query [SEP] chunk [SEP]` encoded *jointly*
Interaction	None until a final cosine between two pooled 384-d vectors	Full self-attention between every query token and every chunk token, in every layer
Pre-computation	Chunk vectors computed once, offline, and indexed	Impossible: the score depends on the pair
Good at	Fast, scalable, topical recall	Precise relevance: negation, exceptions, roles, paraphrase
Weak at	Qualifiers ("not", "only", "except for") barely move a pooled vector; lexical look-alikes score high	$O(N)$ forward passes per query; can only re-order what it is given

In short, the bi-encoder asks *"is this chunk about the same topic?"* and the cross-encoder asks *"does this chunk answer this question?"*

3 Failure-mode analysis

Query selection. About 300 sentences in the regulation contain a prohibition or exception marker ("may not", "forbidden", "except for", "notwithstanding"). From these I wrote about 67 questions in everyday language, avoiding the regulation's own wording, and ran each one through both rankers.

- For about 51 questions the *gold clause* (the clause that actually answers it) was already in the bi-encoder's top 3. Section headers make the baseline strong on topical questions.

- Of the 16 misses, re-ranking **rescued 7** (Q1–Q7 below).
- In 7 more, the gold clause was not in the top 10 at all, which no re-ranker can fix.
- In the remaining 2, re-ranking did not lift it into the top 3.

Gold clauses are identified by a unique phrase, not by a hard-coded chunk ID.

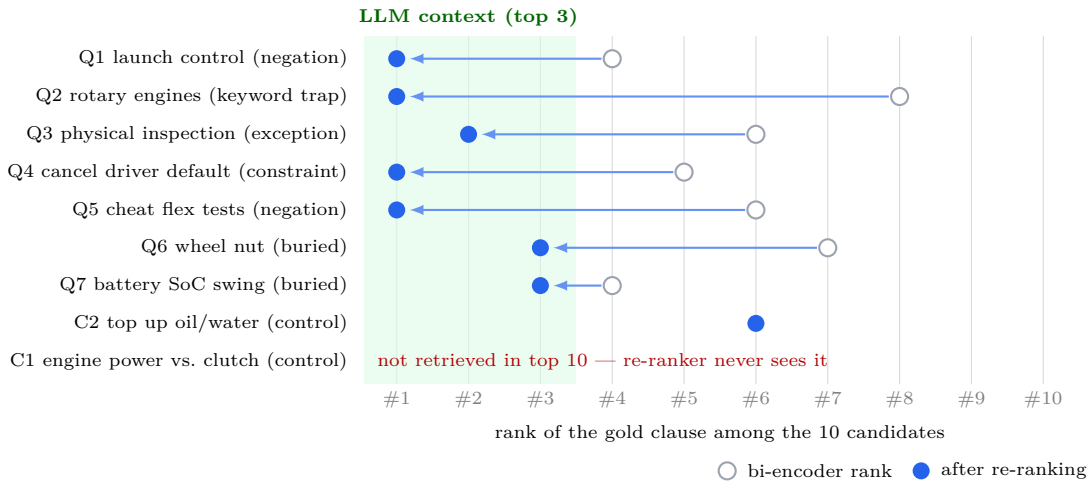


Figure 1: Rank of the gold clause before (hollow) and after (filled) cross-encoder re-ranking. Each of the 7 failure queries moves from rank 4–8 into the top 3, five of them to #1.

	Gold clause	**Shift**	**Baseline top 3 contained instead…**	**Answer 1 (baseline) → Answer 2 (re-ranked)**
Q1	C8.3.1 start systems	#4→#1	three *lights* clauses (signal display, ERS lights, rear lights)	"not enough information" → **No**: any system detecting the start signal is not permitted
Q2	C5.1.1 engine spec.	#8→#1	*rotating* parts: MGU-K, rev limits, "reciprocating and rotating" materials	"not enough information" → **No**: only 4-stroke reciprocating-piston engines
Q3	C1.5 duty of compliance	#6→#2	sibling C1.5/C3.3 chunks on software and CAD checks	"No" for the wrong reason (cites electronics rule) → "No", quoting *"except for safety features"*
Q4	C8.2.4 control electronics	#5→#1	C9.7 Reverse gear ("by the *driver* at any time"), start systems	"not enough information" → **No**: may not be turned off before the end of the lockout
Q5	C3.18.1 flexibility tests	#6→#1	C3.2.2 rigidity rule, impact *test* procedures	"not enough information" → **No**: circumventing the tests is prohibited
Q6	C10.9.1 wheel retention	#7→#3	C3.14 wheel bodywork geometry, fastener tests	"not enough information" → **No**: fastener may mount only the wheel
Q7	C5.2.9 energy flow	#4→#3	sibling C5.2 limits (8.5 MJ recharge/lap)	quotes the *wrong* limits → **4 MJ** state-of-charge window
C1	C5.13.3 PU control	—	clutch-control clauses only	"not enough information" → wrong **"Yes"**
C2	C4.4 adding during race	#6→#6	oil-sampling clauses (C16)	"not enough information" in both

Table 1: Rank shifts and answer comparison (identical prompt; only the three context chunks differ).

Why the bi-encoder failed.

- **Keyword and morphology traps (Q1, Q2, Q4).** The pooled vector latches onto one salient word: "lights", "rotary"≈"rotating", "driver". The cross-encoder scored these distractors at logits between −5 and −11.
- **Abstract prohibitions (Q1, Q5, Q6).** The regulation never says "launch control" or "cheat"; it says "any system … the purpose of which is to detect when a race start signal is given" and "circumventing the tests". Only joint attention connects the user's intent to that wording.

- **Exceptions inside general clauses (Q3).** The decisive words, "except for safety features", sit mid-paragraph in C1.5.
- **Crowded sections (Q7).** C5.2 is split into many chunks that all quote MJ limits.

Effect on the answers. The baseline produced a usable answer for **0 of 7** failure queries:

- 5 were refusals;
- 1 reached the right verdict for the wrong reason (Q3);
- 1 quoted the wrong limit (Q7).

The re-ranked pipeline was correct and cited the governing clause in **7 of 7**. Because of the strict grounding prompt, a missing clause mostly shows up as a *refusal*: the user is told the rules are silent on something they explicitly prohibit.

Limits. The controls show that re-ranking is capped by stage-1 recall. In C1, the true clause was never retrieved, and the cross-encoder promoted related clutch clauses that led the LLM to a confident *wrong* "Yes". Margins can also be thin: in Q4 the gold clause won by only 0.1 logits.

4 Latency profiling

Measured on a laptop CPU (4 threads): 20 repeats × 9 queries for the retrieval stages, plus the 9 real LLM calls per pipeline.

Stage	**Pipeline 1**	**Pipeline 2**
Query embedding (bi-encoder)	6.6 ± 0.5 ms	6.6 ± 0.5 ms
FAISS exact search (1,032 × 384-d)	0.1 ms	0.1 ms
Cross-encoder, 10 pairs (one batch)	—	31.6 ± 2.8 ms
Retrieval total	**6.6 ms**	**37.7 ms** (≈5.7×)
LLM call (DeepSeek API, network)	1379 ± 180 ms	1184 ± 169 ms
End-to-end (mean LLM time)	**≈1.29 s**	**≈1.32 s** (+2.4%)

Table 2: Per-stage latency (mean ± std). Offline indexing of all chunks took 2.1 s.

Measured against retrieval alone, re-ranking is expensive: 10 cross-attention passes at about 3.2 ms each, versus one embedding and a sub-millisecond lookup. Measured against the whole request it is negligible: the +31 ms is smaller than the ±170 ms jitter of the LLM call. For roughly 2% more latency, the stress-test set went from 0/7 to 7/7 correctly grounded answers. The LLM difference between the two pipelines is network noise.

5 Scaling: why not cross-encode the whole collection?

- **Bi-encoder.** Chunk embeddings do not depend on the query, so they are computed *once, offline* and indexed. A query then costs one encoder pass plus a nearest-neighbour search, which ANN indexes (HNSW, IVF) make roughly $O(\log N)$.
- **Cross-encoder.** Its score comes from attention over the *concatenated* pair, so nothing can be precomputed or indexed. Every query costs N full transformer passes, each quadratic in sequence length: $O(N)$ with a large constant.

I measured this directly. Cross-encoding *all* 1,032 chunks for one query took **2.4 s** (2.35 ms per pair), about **64**× the 38 ms of the two-stage pipeline. Extrapolating the measured cost per pair:

Corpus size (chunks)	1,032 (this doc)	10,000	100,000	10,000,000
Cross-encode everything, per query	2.4 s	23.5 s	3.9 min	6.5 h
Two-stage (top-10 retrieve + re-rank)	38 ms	≈38 ms + ANN	≈38 ms + ANN	≈38 ms + ANN

A GPU would shrink these numbers by about 10–100×, but the cost would still grow linearly with the corpus, and most of the compute would go on obviously irrelevant clauses. The **retrieve-then-re-rank** design gets the best of both: the cheap, high-*recall* bi-encoder narrows N chunks to k candidates, and the expensive, high-*precision* cross-encoder reads only those k, so its cost depends on k, not N. The trade-off, shown by control C1, is that the final quality is capped by stage-1 recall. In practice k is tuned (often 20–100), and stage 1 is strengthened with hybrid BM25 + dense retrieval.

6 Conclusion

On a 254-page technical regulation, a bi-encoder alone retrieves the right *topic* but often not the right *clause*. Its failures are negations, exceptions, abstract prohibitions and morphological look-alikes. Re-ranking the same 10 candidates with a cross-encoder moved the gold clause from rank 4–8 into the top 3 in every rescued case, and turned refusals and wrong limits into correct, cited answers. The cost was about 31 ms per query, roughly 2% of end-to-end latency. Re-ranking does not fix recall, so stage 1 still decides what can be found.

Repository: `https://github.com/XProtocol0/rag-baseline-vs-reranking`